

Main Report

Thông tin sinh viên

Mục	Giá trị
Họ tên sinh viên:	Trần Minh Quang
MSSV:	23127464
Lớp / Khoá:	CS423 / CSC13003
Mã bài tập :	HW02
Ngày làm bài:	27-06-2026
Công cụ AI đã dùng:	ChatGPT, Grok, Claude , Antigravity
Repository:	CS423-CSC15003-Testing-N08 - Branch: 23127464

Pool A

FR-05: Xem danh sách và Tìm kiếm sản phẩm

1. Tổng quan

FR-05 yêu cầu hệ thống EShop hiển thị danh sách sản phẩm dạng lưới (grid) trên trang chủ và cung cấp chức năng tìm kiếm sản phẩm theo tên. Các yêu cầu cụ thể bao gồm:

- Hiển thị danh sách sản phẩm dạng grid với: Ảnh (tỷ lệ chuẩn, có alt text), Tên sản phẩm, Giá (đơn vị đ, định dạng phân cách hàng nghìn).
- Thanh tìm kiếm tìm theo tên sản phẩm, từ khóa tìm kiếm phải được hiển thị an toàn (không render HTML).
- Trạng thái loading khi đang tải dữ liệu.
- Thông báo empty state khi không có kết quả.
- Trang chủ chỉ có đúng một thẻ `<h1>`.

Các tài liệu đặc tả được sử dụng làm cơ sở thiết kế test case:

- [description_project.md](#) -- Mục FR-05 (dòng 73-81)
- [api_specification.md](#) -- API 3.1: `GET /api/products?search=keyword`

2. Domain Testing

2.1. Quy trình áp dụng kỹ thuật Domain Testing

Kỹ thuật Domain Testing được áp dụng theo quy trình 3 bước như sau:

Bước 1 -- Xác định biến đầu vào (Input Variables)

Từ phân tích đặc tả FR-05 trong `description_project.md` và `api_specification.md`, xác định được **1 biến đầu vào duy nhất** từ phía người dùng:

#	Biến	Kiểu	Nguồn	Mô tả
V1	<code>search_keyword</code>	String	Thanh tìm kiếm (UI) / Query string ? <code>search=keyword</code> (API)	Từ khóa tìm kiếm sản phẩm theo tên

Ngoài ra, xác định thêm **8 yêu cầu UI / Hành vi hệ thống** (implicit variables) mà hệ thống phải đáp ứng. Đây không phải biến đầu vào của người dùng, mà là các điều kiện hệ thống cần đảm bảo:

#	Yêu cầu	Nguồn đặc tả	Loại kiểm tra
UI-1	Danh sách sản phẩm hiển thị dạng lưới (grid)	FR-05	UI Layout
UI-2	Mỗi SP hiển thị: Ảnh (tỷ lệ chuẩn, có alt text), Tên SP, Giá (đ, phân cách hàng nghìn)	FR-05	UI Content
UI-3	Từ khóa tìm kiếm phải hiển thị an toàn (không render HTML)	FR-05, SEC-04	Security / XSS
UI-4	Khi đang tải dữ liệu phải hiển thị trạng thái loading	FR-05	UI State
UI-5	Không có kết quả tìm kiếm phải hiển thị empty state phù hợp	FR-05	UI State
UI-6	Trang chủ có đúng một thẻ <code><h1></code>	FR-05	DOM Structure
UI-7	Ảnh sản phẩm phải có thuộc tính alt mô tả nội dung (không rỗng)	FR-05	Accessibility
UI-8	Đơn vị tiền: ký hiệu đ, định dạng phân cách hàng nghìn	FR-05	Format

Bước 2 -- Phân hoạch tương đương (Equivalence Partitioning)

Áp dụng EP cho biến `search_keyword` (kiểu String). Chia miền dữ liệu thành 7 phân hoạch (Equivalence Classes), bao gồm cả phân hoạch Malicious Payload để kiểm tra bảo mật:

Partition ID	Phân hoạch (Equivalence Class)	Giá trị đại diện	Kết quả mong đợi
EP1	Rỗng (Empty/Blank)	<code>""</code> (chuỗi rỗng)	Hiển thị toàn bộ danh sách sản phẩm
EP2	Từ khóa hợp lệ -- có kết quả	<code>"Iphone"</code> (từ khóa khớp tên SP)	Hiển thị danh sách sản phẩm có tên chứa từ khóa

Partition ID	Phân hoạch (Equivalence Class)	Giá trị đại diện	Kết quả mong đợi
EP3	Từ khóa hợp lệ -- không có kết quả	"xyznoexist123" (không khớp)	Hiển thị thông báo empty state phù hợp
EP4	Từ khóa có ký tự đặc biệt	"Áo @#\$%"	Hệ thống xử lý an toàn, hiển thị kết quả hoặc empty state mà không bị lỗi
EP5	Chỉ khoảng trắng (Whitespace-only)	" " (3 dấu cách)	Hệ thống xử lý như chuỗi rỗng hoặc trả kết quả phù hợp, không bị lỗi
EP6	Malicious Payload -- XSS	<script>alert('XSS')</script>	Hệ thống hiển thị an toàn chuỗi dạng plain text, KHONG render HTML/JS
EP7	Malicious Payload -- SQL Injection	' OR '1'='1' --	Hệ thống xử lý an toàn, KHONG trả về toàn bộ sản phẩm bất thường

Bước 3 -- Tổng hợp Domain Matrix và tạo Test Case

Từ 7 phân hoạch EP và 7 yêu cầu UI (trong đó 2 yêu cầu UI-3 và UI-7 đã được bao phủ bởi EP6 và được gộp/loại sau human review), tổng hợp thành **14 test case ban đầu** (7 EP + 7 UI). Sau khi human review, giảm còn **12 test case** do loại bỏ 2 test case trùng lặp/ngoài phạm vi (DT-013: alt text -- nguồn gốc từ FR-24, DT-014: format giá -- trùng với DT-012).

Ma trận tổng hợp cuối cùng:

TC ID	Loại	Phân hoạch / Yêu cầu	Giá trị search_keyword	Kết quả mong đợi
DT-001	EP	EP1 -- Rỗng	"" (rỗng)	Hiển thị toàn bộ danh sách sản phẩm
DT-002	EP	EP2 -- Hợp lệ, có kết quả	"Iphone"	Hiển thị danh sách SP có tên chứa "Iphone"
DT-003	EP	EP3 -- Hợp lệ, không có kết quả	"xyznoexist123"	Hiển thị empty state phù hợp
DT-004	EP	EP4 -- Ký tự đặc biệt	"Áo @#\$%"	Xử lý an toàn, không lỗi hệ thống
DT-005	EP	EP5 -- Chỉ whitespace	" "	Xử lý phù hợp (như rỗng hoặc trả kết quả), không lỗi
DT-006	EP	EP6 -- XSS Payload	<script>alert('XSS')</script>	Hiển thị an toàn dạng plain text, KHONG render HTML/JS

TC ID	Loại	Phân hoạch / Yêu cầu	Giá trị <code>search_keyword</code>	Kết quả mong đợi
DT-007	EP	EP7 -- SQL Injection	' OR '1'='1' --	Xử lý an toàn, KHÔNG trả về toàn bộ SP bất thường
DT-008	UI	UI-1: Grid layout	N/A	Layout dạng lưới (grid/flex-wrap)
DT-009	UI	UI-2: Card SP (Ảnh + Tên + Giá)	N/A	Đủ 3 thành phần trên mỗi product card, giá hiển thị ký hiệu đ
DT-010	UI	UI-4: Loading state	N/A	Có loading indicator khi đang fetch dữ liệu
DT-011	UI	UI-6: Đúng 1 thẻ <code><h1></code>	N/A	DOM chỉ chứa đúng 1 element <code><h1></code>
DT-012	UI	UI-8: Format giá đ	N/A	Giá hiển thị đúng định dạng: ký hiệu đ + phân cách hàng nghìn

2.2. Kết quả thực thi

TC ID	Tên Test Case	Kết quả	Ghi chú
TC-FR05-DT-001	Tìm kiếm với từ khóa rỗng	Passed	
TC-FR05-DT-002	Tìm kiếm từ khóa hợp lệ có kết quả	Passed	
TC-FR05-DT-003	Tìm kiếm từ khóa không có kết quả	Failed	Không hiển thị empty state message
TC-FR05-DT-004	Tìm kiếm từ khóa có ký tự đặc biệt	Failed	Không trả về kết quả hoặc empty state
TC-FR05-DT-005	Tìm kiếm chỉ khoảng trắng	Passed	
TC-FR05-DT-006	Tìm kiếm với XSS Payload	Failed	Server trả 500, lộ raw DB error
TC-FR05-DT-007	Tìm kiếm với SQL Injection	Failed	SQL Injection thành công, trả về toàn bộ SP
TC-FR05-DT-008	Danh sách SP hiển thị dạng grid	Passed	
TC-FR05-DT-009	Card SP hiển thị Ảnh + Tên + Giá	Failed	Giá hiển thị 'VND' thay vì ký hiệu đ

TC ID	Tên Test Case	Kết quả	Ghi chú
TC-FR05-DT-010	Trạng thái loading khi đang tải	Failed	Không có loading indicator
TC-FR05-DT-011	Trang chủ có đúng 1 thẻ h1	Failed	Có 2 thẻ h1 thay vì 1
TC-FR05-DT-012	Giá hiển thị đúng format đ	Failed	Hiển thị 'VND' thay vì ký hiệu đ

Thống kê: 4 Passed (33.3%) / 8 Failed (66.7%) trên tổng số 12 test case.

2.3. Các lỗi phát hiện từ Domain Testing

Bug ID	TC liên quan	Mô tả ngắn	Mức độ
BUG-FR05-001	DT-003	Thiếu empty state khi không có kết quả tìm kiếm	Minor
BUG-FR05-002	DT-004	Tìm kiếm ký tự đặc biệt không trả kết quả/empty state	Minor
BUG-FR05-003	DT-006	XSS payload gây lỗi 500, lộ raw DB error	Critical
BUG-FR05-004	DT-007	SQL Injection thành công, trả về toàn bộ sản phẩm	Critical
BUG-FR05-005	DT-009, DT-012	Ký hiệu tiền tệ hiển thị 'VND' thay vì đ	Minor
BUG-FR05-006	DT-010	Thiếu loading indicator khi đang tải dữ liệu	Minor
BUG-FR05-007	DT-011	Trang chủ có 2 thẻ h1 thay vì 1	Trivial

3. Boundary Value Analysis (BVA)

3.1. Quy trình áp dụng kỹ thuật BVA

Kỹ thuật Boundary Value Analysis (BVA) được áp dụng theo quy tắc nghiêm ngặt (STRICT BVA RULE) được định nghĩa trong file [CLAUDE.md](#):

BVA chỉ được áp dụng cho các biến số (numerical variables) như price, quantity, total_amount. KHÔNG được ép hoặc suy diễn BVA trên các biến phi số như String (search queries, emails), Categorical data (roles, statuses), hoặc UI/DOM properties.

Bước 1 -- Đánh giá khả năng áp dụng BVA

Xét tất cả biến đầu vào của FR-05:

Biến	Kiểu	Có phải biến số (numerical) không?	Áp dụng BVA?
search_keyword	String	Không	Không

Bước 2 -- Kết luận

FR-05 chỉ có 1 biến đầu vào là `search_keyword` thuộc kiểu String. Theo STRICT BVA RULE, BVA chỉ áp dụng cho biến số (numerical). Do đó:

"No numerical variables found. BVA is skipped."

Biến `search_keyword` đã được bao phủ đầy đủ bởi kỹ thuật Equivalence Partitioning (EP) với 7 phân hoạch ở phần Domain Testing phía trên.

3.2. Giải thích lý do không áp dụng BVA cho FR-05

Trong bối cảnh FR-05, chức năng tìm kiếm sản phẩm nhận duy nhất một biến đầu vào là chuỗi ký tự (String). Các ranh giới của biến chuỗi (ví dụ: độ dài tối thiểu, độ dài tối đa) không được đặc tả ràng buộc cụ thể trong tài liệu `description_project.md` hoặc `api_specification.md`. Vì vậy:

- Không tồn tại điểm ranh giới số học rõ ràng để áp dụng BVA (ví dụ: không có ràng buộc "từ khóa phải từ 1 đến 255 ký tự").
- Các trường hợp biên của chuỗi rỗng (""), chỉ khoảng trắng (" "), chuỗi có ký tự đặc biệt đã được xử lý qua các phân hoạch EP tương ứng (EP1, EP5, EP4).
- Các trường hợp bảo mật (XSS, SQL Injection) cũng đã được bao phủ qua phân hoạch EP6 và EP7.

Do đó, việc áp dụng BVA cho FR-05 là không phù hợp và không tạo thêm giá trị kiểm tra bổ sung.

4. Quy trình áp dụng CLAUDE.md cho AI Agent để tạo test case

4.1. Giới thiệu về CLAUDE.md

File `CLAUDE.md` là file cấu hình hướng dẫn cho AI Agent (Antigravity - Claude Opus 4.6 Thinking) hoạt động như một ISTQB-Certified QA Test Designer. File này định nghĩa:

- **Vai trò:** QA Test Designer chuyên về Black-Box Testing
- **Ràng buộc:** Hành động từng bước, dừng lại và chờ phê duyệt sau mỗi bước
- **Nguồn dữ liệu:** Chỉ dựa trên `description_project.md` và `api_specification.md`
- **Quy tắc BVA:** STRICT BVA RULE -- chỉ áp dụng cho biến số (numerical)
- **Cấu trúc file:** Định dạng và đường dẫn chuẩn cho test cases, bug reports, test runs, gap analysis
- **Templates:** 2 template chuẩn cho Domain Testing (EP) và BVA
- **Workflow:** 5 bước từ phân tích đến báo cáo lỗi

4.2. Quy trình thực hiện chi tiết

Quy trình áp dụng CLAUDE.md được thực hiện qua **3 giai đoạn chính** với sự tương tác giữa người dùng và AI Agent:

Giai đoạn 1: Phân tích và Thiết kế (Steps 1-2-3 trong CLAUDE.md)

1. Người dùng cung cấp prompt khởi động quá trình QA với cấu hình cụ thể:

- `[FR-DIR] = FR-05-search`
- `[FR-ID] = FR05`
- Yêu cầu bắt buộc có Malicious Payload (XSS, SQL Injection)

2. AI Agent đọc và phân tích 2 file đặc tả ([description_project.md](#) tại mục FR-05, [api_specification.md](#) tại API 3.1), xác định:
 - 1 biến đầu vào: [search_keyword](#) (String)
 - 8 yêu cầu UI/hành vi hệ thống (UI-1 đến UI-8)
3. AI Agent áp dụng Equivalence Partitioning, chia miền dữ liệu của [search_keyword](#) thành 7 phân hoạch (EP1-EP7), bao gồm 2 phân hoạch Malicious Payload theo yêu cầu.
4. AI Agent đánh giá khả năng áp dụng BVA theo STRICT BVA RULE. Kết luận: [search_keyword](#) là String, không phải numerical -- BVA bị bỏ qua.
5. AI Agent trình bày bảng phân tích logic (bảng biến, EP, BVA) và dừng lại chờ người dùng phê duyệt trước khi tạo test case.

Kết quả giai đoạn này được lưu tại: [implementation_plan_FR05.md](#)

Giai đoạn 2: Tạo Test Case (Step 4 trong CLAUDE.md)

1. Sau khi người dùng phê duyệt bảng phân tích, AI Agent tạo 14 file test case (7 EP + 7 UI) theo **Template 1 (Domain Testing)** được định nghĩa trong CLAUDE.md.
2. AI Agent sử dụng 2 subagent song song để tăng tốc:
 - EP Test Case Writer: tạo DT-001 đến DT-007 (Equivalence Partitioning)
 - UI Test Case Writer: tạo DT-008 đến DT-014 (UI Requirements)
3. AI Agent kiểm tra và dọn dẹp: phát hiện 5 file thừa (DT-015 đến DT-019) do subagent tạo thêm và xóa chúng, giữ lại đúng 14 file.
4. AI Agent dừng lại và yêu cầu người dùng thực thi test case trên hệ thống thực (manual testing).

Giai đoạn 3: Thực thi, Human Review, và Báo cáo (Step 5 trong CLAUDE.md)

1. Người dùng thực thi 14 test case trên hệ thống EShop thực tế và cập nhật actual result + status cho từng file.
2. Trong quá trình manual testing, người dùng nhận ra và thực hiện các điều chỉnh:
 - **Loại bỏ 2 test case** (DT-013 về alt text, DT-014 về format giá) vì ngoài phạm vi FR-05 hoặc trùng lặp.
 - **Điều chỉnh test data**: đổi từ khóa "[Áo](#)" (AI đoán) thành "[Iphone](#)" (sản phẩm thực tế trong DB).
3. Người dùng báo cáo kết quả: 4 Passed (DT-001, DT-002, DT-005, DT-008) và 8 Failed.
4. AI Agent nhận kết quả và triển khai Step 5:
 - Tạo [FR-05-search-run.md](file:///e:/Testing/CS423-CSC15003-Testing-N08/tests/test-runs/FR-05-search-run.md): tổng hợp kết quả test run
 - Tạo 7 bug reports ([BUG-FR05-001](file:///e:/Testing/CS423-CSC15003-Testing-N08/bug-reports/BUG-FR05-001.md) đến [BUG-FR05-007](file:///e:/Testing/CS423-CSC15003-Testing-N08/bug-reports/BUG-FR05-007.md)): theo template GitHub Issue

- ### 4.3. Sơ đồ quy trình tổng quát

8 / 44

- AI tự động đọc thêm các requirement khác (FR-21, FR-24) dù người dùng chỉ yêu cầu tập trung vào FR-05.
- Kết quả: Tạo ra test case DT-013 (alt text -- nguồn từ FR-24) và DT-014 (format giá -- trùng với DT-012 từ FR-21) -- cả hai bị người dùng loại bỏ.

Hạn chế 2: Test data lệch với thực tế

- AI chỉ được đọc đặc tả (Black-box) nên không biết dữ liệu thực tế trong CSDL.
- Kết quả: AI dùng từ khóa "Áo" làm test data cho EP2, nhưng trong CSDL thực tế không có sản phẩm tên "Áo". Người dùng phải điều chỉnh thành "Iphone".

Hạn chế 3: Test case trùng lặp

- AI truy vết (traceability) giữa các requirement liên quan, dẫn đến test case bị phân tán và trùng lặp.
- Kết quả: Cả DT-009 (UI-2: Card SP) và DT-012 (UI-8: Format giá) đều phát hiện cùng một lỗi (VND thay vì đ).

4.6. Đánh giá tổng thể

Tiêu chí	Đánh giá
Số lượng test case AI tạo ban đầu	14
Số lượng test case sau human review	12 (loại 2)
Số lượng test case phát hiện lỗi	8/12 (66.7%)
Số lượng bug phát hiện	7 (2 Critical, 4 Minor, 1 Trivial)
Độ chính xác của test design	Cao -- phủ được các vùng quan trọng (XSS, SQLi, UI)
Cần chỉnh sửa bởi người dùng	Test data, phạm vi test case, loại bỏ trùng lặp

AI Agent là công cụ hữu ích cho việc thiết kế test case ban đầu, nhưng **human review là bắt buộc** để:

- Điều chỉnh phạm vi test case cho đúng requirement được yêu cầu
- Thay thế test data giả định bằng dữ liệu thực tế của hệ thống
- Loại bỏ test case trùng lặp hoặc ngoài phạm vi
- Xác nhận kết quả test trên hệ thống thực

Pool B

FR-08: Thanh toán (Checkout)

1. Tổng quan

FR-08 định nghĩa quy trình thanh toán (Checkout) của hệ thống EShop. Các yêu cầu cụ thể bao gồm:

- Chỉ người dùng **đã đăng nhập** mới tiến hành thanh toán được.
- **Tổng tiền thanh toán** được tính tự động từ giỏ hàng và không cho phép người dùng chỉnh sửa trực tiếp.
- Giao diện hiển thị đầy đủ danh sách sản phẩm đặt mua.

- **Backend phải tự tính lại tổng tiền; không chấp nhận giá trị `total_amount` do client gửi lên.**
- Sau thanh toán thành công, giỏ hàng được xóa.

Các tài liệu đặc tả được sử dụng làm cơ sở thiết kế test case:

- [description_project.md](#) -- Mục FR-08 (dòng 102-108)
- [api_specification.md](#) -- Mục 4.3: Đặt hàng (dòng 129-137)

Phát hiện quan trọng: Có mâu thuẫn giữa đặc tả FR-08 và API Specification. FR-08 yêu cầu "*Backend phải tự tính lại tổng tiền; không chấp nhận giá trị `total_amount` do client gửi lên*", nhưng API Spec lại cho phép client gửi `total_amount` trong body của `POST /api/checkout`. Đây là **attack surface** quan trọng: hacker có thể dùng Postman gửi `total_amount` ảo để kiểm tra backend có thực sự bỏ qua giá trị này hay không.

Môi trường kiểm thử: API testing qua Postman trên `http://localhost:3000`.

2. Domain Testing

2.1. Quy trình áp dụng kỹ thuật Domain Testing

Kỹ thuật Domain Testing được áp dụng theo quy trình 3 bước như sau:

Bước 1 -- Xác định biến đầu vào (Input Variables)

Từ phân tích FR-08 và API Specification, xác định được **4 biến đầu vào**:

#	Biến	Kiểu	Nguồn	Miền giá trị / Ràng buộc
V1	<code>Authorization</code> (JWT Token)	String (Header)	HTTP Header	Token JWT hợp lệ do hệ thống cấp sau đăng nhập
V2	<code>total_amount</code>	Numeric (Body)	Request Body (JSON)	Theo FR-08: Backend tự tính, KHÔNG tin client. Nhưng API cho phép gửi -- attack vector
V3	<code>shipping_address</code>	String (Body)	Request Body (JSON)	Địa chỉ giao hàng, chuỗi ký tự
V4	Cart State	Implicit (Server)	Server-side (DB)	Giỏ hàng phải có ít nhất 1 sản phẩm

Bước 2 -- Phân hoạch tương đương (Equivalence Partitioning)

Áp dụng EP cho từng biến:

V1: Authorization (JWT Token)

EP ID	Partition	Mô tả	Expected
EP-V1-01	Valid -- Token hợp lệ	Bearer token JWT còn hạn, đúng user	Cho phép checkout
EP-V1-02	Invalid -- Không có token	Không gửi header Authorization	401 Unauthorized

EP ID	Partition	Mô tả	Expected
EP-V1-03	Invalid -- Token sai/hết hạn	Token rác hoặc hết hạn	401/403

V2: total_amount (Attack Vector -- Client gửi ảo)

Tư duy Black-box Hacker: Dù FR-08 nói backend tự tính, API spec vẫn accept `total_amount` từ client. Hacker dùng Postman có thể gửi bất kỳ giá trị nào. Cần kiểm tra backend có THAT SU bỏ qua giá trị này không.

EP ID	Partition	Giá trị mẫu	Expected (theo FR-08)
EP-V2-01	Giá trị đúng -- Khớp tổng thực tế	200000	Checkout thành công, tổng = backend tự tính
EP-V2-02	Giá trị thấp hơn thực tế	1	Backend bỏ qua, tổng = backend tự tính
EP-V2-03	Giá trị = 0 -- mua miễn phí	0	Backend bỏ qua, tổng = backend tự tính
EP-V2-04	Giá trị âm -- tạo refund	-50000	Backend bỏ qua, tổng = backend tự tính
EP-V2-05	Giá trị quá cao -- over-charge	99999999	Backend bỏ qua, tổng = backend tự tính
EP-V2-06	Không gửi trường	(omit)	Backend tự tính, checkout thành công
EP-V2-07	Kiểu dữ liệu sai	"abc"	Backend reject hoặc bỏ qua, tự tính

V3: shipping_address

EP ID	Partition	Giá trị mẫu	Expected
EP-V3-01	Valid -- Địa chỉ hợp lệ	"123 Le Loi, Q1, TP.HCM"	Checkout thành công
EP-V3-02	Empty -- Chuỗi rỗng	""	Lỗi validation
EP-V3-03	Missing -- Không gửi trường	(omit)	Lỗi hoặc dùng địa chỉ mặc định
EP-V3-04	XSS Payload	"<script>alert('xss')</script>"	Lưu nhưng escape khi hiển thị (SEC-04)
EP-V3-05	SQL Injection	"'; DROP TABLE orders;--"	Parameterized query ngăn chặn (SEC-05)

V4: Cart State

EP ID	Partition	Mô tả	Expected
EP-V4-01	Non-empty -- Có sản phẩm	Cart co >= 1 item	Checkout thành công
EP-V4-02	Empty -- Giỏ hàng trống	Cart = 0 items	Không cho checkout

Bước 3 -- Tổng hợp Domain Matrix và tạo Test Case

Tổng hợp 16 EP thành **15 test case Domain Testing** (DT-001 đến DT-015), bao gồm 1 test case post-condition:

TC ID	Biến chính	Mô tả
DT-001	V1=Valid, V2=đúng, V3=valid, V4=non-empty	Checkout thành công (happy path)
DT-002	V1=Missing	Checkout thiếu Token (chưa đăng nhập)
DT-003	V1=Invalid	Checkout Token sai/hết hạn/malformed
DT-004	V2=EP-V2-02	Hacker gửi <code>total_amount</code> thấp hơn thực tế (1)
DT-005	V2=EP-V2-03	Hacker gửi <code>total_amount</code> = 0 (mua miễn phí)
DT-006	V2=EP-V2-04	Hacker gửi <code>total_amount</code> âm (-50000)
DT-007	V2=EP-V2-05	Hacker gửi <code>total_amount</code> quá cao (99999999)
DT-008	V2=EP-V2-06	Không gửi trường <code>total_amount</code>
DT-009	V2=EP-V2-07	Gửi <code>total_amount</code> kiểu string ("abc")
DT-010	V3=EP-V3-02	<code>shipping_address</code> rỗng
DT-011	V3=EP-V3-03	Thiếu trường <code>shipping_address</code>
DT-012	V3=EP-V3-04	XSS Injection trong <code>shipping_address</code>
DT-013	V3=EP-V3-05	SQL Injection trong <code>shipping_address</code>
DT-014	V4=EP-V4-02	Checkout với giỏ hàng trống
DT-015	Post-condition	Giỏ hàng xóa sau checkout thành công

2.2. Kết quả thực thi Domain Testing

TC ID	Tên Test Case	Kết quả	Bug ID
DT-001	Checkout thành công (happy path)	Failed	BUG-FR08-005
DT-002	Checkout thiếu Token (chưa đăng nhập)	Passed	--
DT-003	Checkout Token sai/hết hạn/malformed	Passed	--
DT-004	Hacker gửi <code>total_amount</code> thấp hơn thực tế	Failed	BUG-FR08-001
DT-005	Hacker gửi <code>total_amount</code> = 0 (mua miễn phí)	Failed	BUG-FR08-001
DT-006	Hacker gửi <code>total_amount</code> âm (hoàn tiền)	Failed	BUG-FR08-001

TC ID	Tên Test Case	Kết quả	Bug ID
DT-007	Hacker gửi <code>total_amount</code> quá cao (over-charge)	Failed	BUG-FR08-001
DT-008	Không gửi trường <code>total_amount</code>	Failed	BUG-FR08-001
DT-009	<code>total_amount</code> kiểu string ("abc")	Failed	BUG-FR08-001
DT-010	<code>shipping_address</code> rỗng	Failed	BUG-FR08-003
DT-011	Thiếu trường <code>shipping_address</code>	Failed	BUG-FR08-003
DT-012	XSS Injection trong <code>shipping_address</code>	Failed	BUG-FR08-002
DT-013	SQL Injection trong <code>shipping_address</code>	Passed	--
DT-014	Checkout với giỏ hàng trống	Failed	BUG-FR08-004
DT-015	Giỏ hàng xóa sau checkout thành công	Failed	BUG-FR08-005

Thống kê Domain Testing: 3 Passed (20.0%) / 12 Failed (80.0%) trên tổng số 15 test case.

2.3. Các lỗi phát hiện

Bug ID	TC liên quan	Mô tả ngắn	Mức độ
BUG-FR08-001	DT-004 - DT-009, BVA-001 - BVA-003	Backend tin tưởng <code>total_amount</code> từ client, không tự tính lại tổng tiền. Hacker có thể mua hàng trị giá 200,000 dong chỉ với 1 dong	Critical
BUG-FR08-002	DT-012	Stored XSS qua trường <code>shipping_address</code> -- chuỗi <code><script></code> được render thành HTML node trên Admin	Major
BUG-FR08-003	DT-010, DT-011	Thiếu validation cho <code>shipping_address</code> -- chấp nhận chuỗi rỗng và missing, tạo đơn hàng không có địa chỉ giao hàng	Major
BUG-FR08-004	DT-014	Cho phép checkout khi giỏ hàng trống, tạo đơn hàng phantom	Major
BUG-FR08-005	DT-001, DT-015	Giỏ hàng không được xóa sau checkout thành công, vi phạm FR-08	Major

2.4. Phân loại lỗi theo nhóm

Nhóm lỗi	Bug ID	Số TC Failed	Đánh giá
Business Logic -- <code>total_amount</code> manipulation	BUG-FR08-001	6 (DT) + 3 (BVA) = 9	Vi phạm FR-08: backend KHÔNG tự tính tổng tiền

Nhóm lỗi	Bug ID	Số TC Failed	Đánh giá
Security -- XSS	BUG-FR08-002	1	Vi phạm SEC-04: shipping_address không escape HTML
Validation -- shipping_address	BUG-FR08-003	2	Thiếu validation cho trường bắt buộc
Business Logic -- Cart State	BUG-FR08-004	1	Checkout khi giỏ trống, vi phạm logic nghiệp vụ
Post-condition -- Cart cleanup	BUG-FR08-005	2	Vi phạm FR-08: giỏ hàng không xóa sau checkout

3. Boundary Value Analysis (BVA)

3.1. Quy trình áp dụng kỹ thuật BVA

Kỹ thuật Boundary Value Analysis (BVA) được áp dụng theo quy tắc nghiêm ngặt (STRICT BVA RULE) được định nghĩa trong file [CLAUDE.md](#):

BVA chỉ được áp dụng cho các biến số (numerical variables) như price, quantity, total_amount. KHÔNG được ép hoặc suy diễn BVA trên các biến phi số.

Bước 1 -- Đánh giá khả năng áp dụng BVA

Xét tất cả biến đầu vào của FR-08:

Biến	Kiểu	Có phải biến số (numerical) không?	Áp dụng BVA?
Authorization (JWT Token)	String (Header)	Không	Không
total_amount	Numeric (Integer)	Có	Có
shipping_address	String (Body)	Không	Không
Cart State	Categorical (Implicit)	Không	Không

Bước 2 -- Xác định Boundary cho total_amount

Biến total_amount là **strictly numerical** -- thỏa mãn STRICT BVA RULE. Đây là biến đầu tiên trong quá trình test 3 FR (FR-05, FR-12, FR-08) mà BVA thực sự được áp dụng.

Ý nghĩa logic:

- Tổng tiền hợp lệ phải > 0 (vì giỏ hàng phải có ít nhất 1 sản phẩm, mỗi sản phẩm có giá > 0)
- Boundary tại 0: Ranh giới giữa giá trị hợp lệ (dương) và không hợp lệ (0 hoặc âm)

Bảng Boundary Points:

Variable	Constraint	Boundary Type	BVA Points
<code>total_amount</code>	Phải > 0	Min boundary tại 0	-1 (OFF ngoại bên trái), 0 (ON), 1 (OFF ngoại bên phải)

Bước 3 -- Tạo BVA Test Case

TC ID	<code>total_amount</code> (client gửi)	Boundary Point	Cart thực tế	Expected
BVA-001	-1	OFF ngoại bên trái (dưới biên)	Có sản phẩm (tổng = 200,000 dong)	Backend bỏ qua giá trị -1, tự tính = 200,000 dong
BVA-002	0	ON (đúng biên)	Có sản phẩm (tổng = 200,000 dong)	Backend bỏ qua giá trị 0, tự tính = 200,000 dong
BVA-003	1	OFF ngoại bên phải (trên biên)	Có sản phẩm (tổng = 200,000 dong)	Backend bỏ qua giá trị 1, tự tính = 200,000 dong

Cách kiểm chứng: Sau khi gửi `POST /api/checkout` với `total_amount` ảo, gọi `GET /api/orders/my-orders` để kiểm tra `total_amount` thực sự được lưu trong đơn hàng. Nếu giá trị lưu = giá trị client gửi (thay vì giá trị tự tính) -- **BUG bảo mật nghiêm trọng**.

3.2. Kết quả thực thi BVA

TC ID	Tên Test Case	Boundary Point	Kết quả	Bug ID
BVA-001	<code>total_amount</code> = -1	OFF ngoại bên trái	Failed	BUG-FR08-001
BVA-002	<code>total_amount</code> = 0	ON	Failed	BUG-FR08-001
BVA-003	<code>total_amount</code> = 1	OFF ngoại bên phải	Failed	BUG-FR08-001

Thống kê BVA: 0 Passed / 3 Failed -- Tất cả 3 boundary points đều phát hiện backend tin tưởng giá trị client gửi.

3.3. Kết quả chi tiết từ BVA

<code>total_amount</code> gửi	<code>total_amount</code> lưu trong DB	Kết quả
-1	-1 dong	Backend tin client -- đơn hàng âm tiền
0	0 dong	Backend tin client -- mua hàng miễn phí
1	1 dong	Backend tin client -- hacker mua 200,000 dong chỉ với 1 dong

Tất cả 3 BVA test case đều xác nhận BUG-FR08-001 (Critical): Backend **không** tự tính lại tổng tiền mà chấp nhận nguyên giá trị `total_amount` do client gửi.

4. Quy trình áp dụng CLAUDE.md cho AI Agent để tạo test case

4.1. Giới thiệu về CLAUDE.md

File [CLAUDE.md](#) là file cấu hình hướng dẫn cho AI Agent (Antigravity - Claude Opus 4.6 Thinking) hoạt động như một ISTQB-Certified QA Test Designer. Đối với FR-08, các thành phần chính của CLAUDE.md được áp dụng bao gồm:

- **Vai trò:** QA Test Designer chuyên về Black-Box Testing
- **Ràng buộc:** Hành động từng bước, dừng lại và chờ phê duyệt sau mỗi bước
- **Nguồn dữ liệu:** Chỉ dựa trên [description_project.md](#) và [api_specification.md](#)
- **Quy tắc BVA:** STRICT BVA RULE -- biến [total_amount](#) thỏa mãn (numerical) -- BVA DUOC ÁP DỤNG
- **Template sử dụng:** Template 1 (Domain Testing) cho 15 TC + Template 2 (BVA) cho 3 TC
- **Workflow:** 5 bước từ phân tích đến báo cáo lỗi

FR-08 là FR đầu tiên trong 3 FR được test mà BVA thực sự được áp dụng (FR-05, FR-12 đều skip BVA).

4.2. Quy trình thực hiện chi tiết

Quy trình áp dụng CLAUDE.md được thực hiện qua **3 giai đoạn chính**:

Giai đoạn 1: Phân tích và Thiết kế (Steps 1-2-3 trong CLAUDE.md)

1. Người dùng cung cấp prompt khởi động quá trình QA với cấu hình cụ thể:
 - [\[FR-DIR\]](#) = [FR-08-checkout](#)
 - [\[FR-ID\]](#) = [FR08](#)
 - Chỉ định phân tích cả FR-08 trong [description_project.md](#) và API spec ([/api/checkout](#))
 - Yêu cầu áp dụng BVA cho [total_amount](#) vì thỏa mãn STRICT BVA RULE
2. AI Agent đọc và phân tích file đặc tả:
 - [description_project.md](#) tại mục FR-08 (dòng 102-108): 5 yêu cầu checkout
 - [api_specification.md](#) tại mục 4.3 (dòng 129-137): endpoint [POST /api/checkout](#)
3. AI Agent phát hiện **mâu thuẫn giữa FR-08 và API Spec**: FR-08 yêu cầu backend tự tính tổng tiền, nhưng API cho phép client gửi [total_amount](#) -- đây là attack surface.
4. AI Agent xác định:
 - 4 biến đầu vào (V1-V4): Authorization, total_amount, shipping_address, Cart State
 - 16 phân vùng EP -- đặc biệt 7 partition cho [total_amount](#) (hacker scenarios)
 - BVA áp dụng CHỈ cho [total_amount](#) (strictly numerical) -- 3 boundary points: [-1](#), [0](#), [1](#)
5. AI Agent trình bày bảng phân tích logic (implementation plan) và dừng lại chờ phê duyệt.

Giai đoạn 2: Tạo Test Case (Step 4 trong CLAUDE.md)

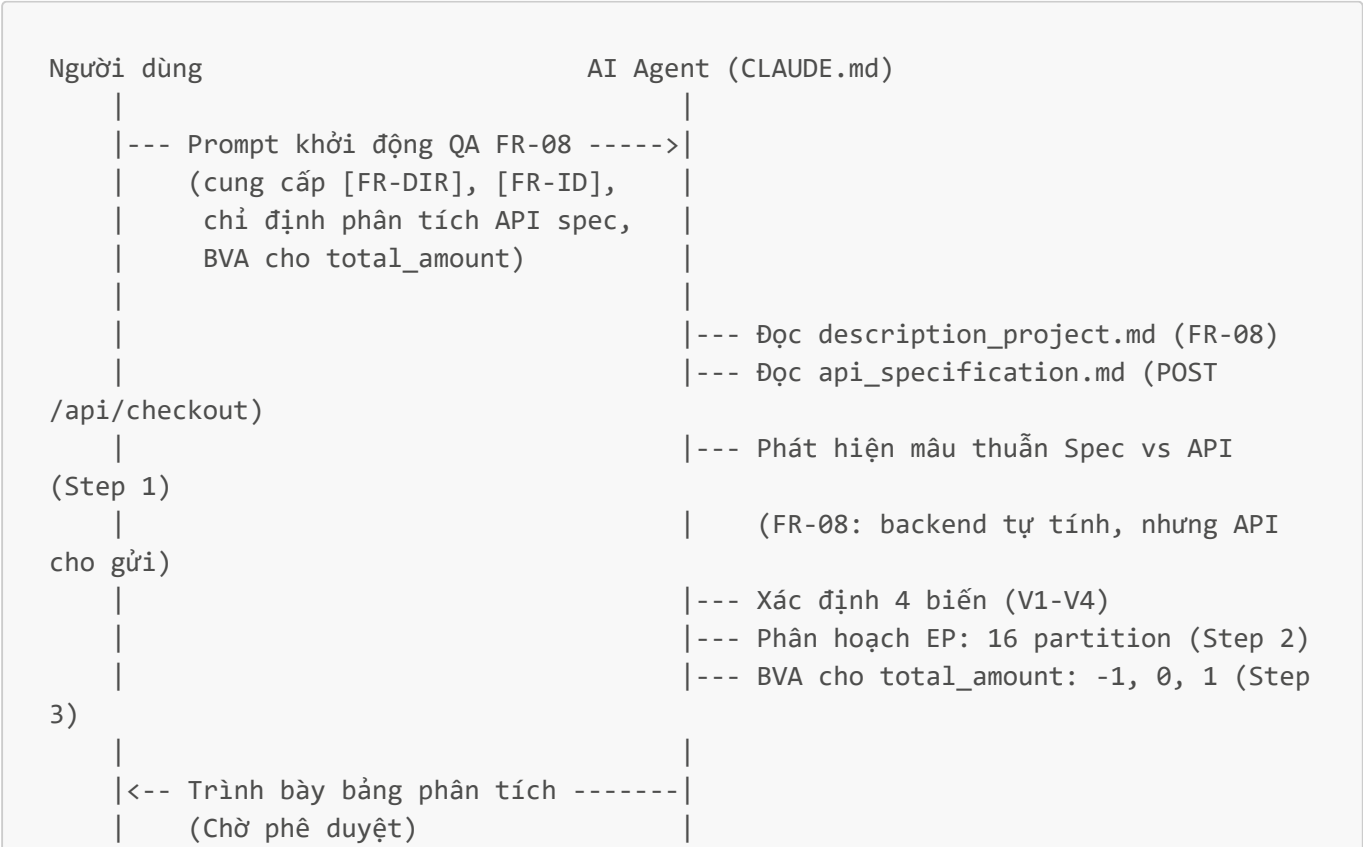
1. Sau khi người dùng phê duyệt bảng phân tích, AI Agent tạo **18 file test case** (15 DT + 3 BVA) theo template CLAUDE.md.
2. AI Agent tạo file theo 3 batch song song:

- Batch 1 (DT-001 đến DT-006): Happy path + Authentication + total_amount manipulation
 - Batch 2 (DT-007 đến DT-012): total_amount continuation + shipping_address validation + Security
 - Batch 3 (DT-013 đến DT-015 + BVA-001 đến BVA-003): SQL Injection + Cart State + Post-condition + BVA boundaries
3. AI Agent xác minh cấu trúc thư mục: `domain-testing/` (15 files) + `bva/` (3 files) -- đây là FR đầu tiên có thư mục `bva/`.
4. AI Agent dừng lại và yêu cầu người dùng thực thi test case trên SUT qua Postman.

Giai đoạn 3: Thực thi, Human Review, và Báo cáo (Step 5 trong CLAUDE.md)

1. Người dùng thực thi 18 test case qua Postman và cập nhật actual result + status cho từng file.
2. Trong quá trình testing, người dùng phát hiện:
- AI tạo test case tương đồng giữa BVA và DT (vd: DT-004 dùng `total_amount = 1` trùng với BVA-003; DT-005 dùng `total_amount = 0` trùng với BVA-002)
 - Đây là vấn đề redundancy mà AI không phát hiện được khi thiết kế
3. Người dùng báo cáo kết quả: 3 Passed / 15 Failed.
4. AI Agent nhận kết quả và phản hồi của người dùng, triển khai Step 5:
- Tạo `FR-08-checkout-run.md`: tổng hợp kết quả test run
 - Tạo 5 bug reports: `BUG-FR08-001.md` đến `BUG-FR08-005.md` -- nhóm theo root cause
 - Tạo `FR-08-checkout-gap-analysis.md`: phân tích trùng lặp DT/BVA

4.3. Sơ đồ quy trình tổng quát



```
|
|
| --- Phê duyệt ----->|
|
|                                     | --- Tạo 15 DT + 3 BVA = 18 files (Step
4)
|                                     | --- Cấu trúc: domain-testing/ + bva/
|
| <--- Yêu cầu test trên SUT -----|
|     qua Postman
|
| --- Thực thi 18 test case ----->|
|     qua Postman trên localhost:3000
|
| --- Báo cáo kết quả:
|     3 Passed / 15 Failed
|     + AI tạo TC trùng lặp giữa
|       DT và BVA (DT-004 = BVA-003,
|       DT-005 = BVA-002) ----->|
|
|                                     | --- Tạo Test Run summary (Step 5)
|                                     | --- Tạo 5 Bug Reports (1 Critical + 4
Major)
|                                     |
|                                     | --- Tạo AI Gap Analysis (DT/BVA
redundancy)
|
| <--- Hoàn thành, yêu cầu commit ----|
|
| --- git add + git commit ----->|
|     (Commit: c71d42f)
```

4.4. Phân tích AI Gap Analysis

Trong quá trình sử dụng AI Agent theo CLAUDE.md cho FR-08, **1 hạn chế chính** được xác định -- đặc biệt liên quan đến BVA:

Hạn chế: Sự trùng lặp giữa Domain Testing (DT) và BVA

Vấn đề được người dùng phát hiện:

BVA Test Case	DT Test Case tương đồng	Giá trị trùng	Vấn đề
BVA-003 (total_amount = 1, OFF ngoại bên phải)	DT-004 (total_amount = 1, EP-V2-02)	1	Cùng giá trị, cùng mục tiêu, chỉ khác label kỹ thuật
BVA-002 (total_amount = 0, ON)	DT-005 (total_amount = 0, EP-V2-03)	0	Cùng giá trị, cùng kịch bản "mua miễn phí"

Nguyên nhân gốc (Root Cause):

- 1. **AI không nhận diện sự chồng lấp logic giữa EP và BVA:** Khi xây dựng EP cho **total_amount**, AI đã tạo các partition riêng cho giá trị 0 (EP-V2-03), giá trị dương nhỏ (EP-V2-02), và giá trị âm (EP-V2-04).

Sau đó, khi áp dụng BVA trên cùng biến, AI lại chọn boundary points tại `{-1, 0, 1}` -- trùng hoàn toàn với các EP đã tạo.

2. **BVA nên bổ sung EP, không lặp lại EP:** Theo nguyên tắc ISTQB, BVA được thiết kế để **bổ sung** cho EP bằng cách tập trung vào ranh giới giữa các partition. Trong trường hợp này, ranh giới tại `0` đã được EP cover rõ ràng nên BVA trở nên dư thừa.
3. **Đặc thù `total_amount` trong FR-08:** Biến này khác biệt so với BVA truyền thống vì FR-08 yêu cầu backend **bỏ qua hoàn toàn** giá trị client gửi. Mọi giá trị của `total_amount` (dương, âm, 0, bất kỳ) đều phải cho **cùng một kết quả**: backend tự tính. BVA mất ý nghĩa khi không có sự phân biệt hành vi tại biên.

Giải pháp cải thiện: Nếu làm lại, AI nên loại bỏ các BVA test case trùng lặp với DT, hoặc sử dụng BVA với giá trị biên khác biệt hơn (ví dụ: `MIN_INT`, `MAX_INT`, `0.001`, `-0.001`).

Chi tiết: [FR-08-checkout-gap-analysis.md](#)

4.5. Đánh giá tổng thể

Tiêu chí	Đánh giá
Số lượng test case AI tạo	18 (15 DT + 3 BVA)
Số lượng test case sau human review	18 (không loại bỏ, nhưng BVA có redundancy với DT)
Số lượng test case phát hiện lỗi	15/18 (83.3%)
Số lượng bug phát hiện	5 (1 Critical, 4 Major)
Độ chính xác của test design	Cao -- phát hiện đúng attack surface (total_amount manipulation), cover security (XSS, SQLi), và post-condition
Cần chỉnh sửa bởi người dùng	Loại bỏ redundancy giữa DT và BVA

So sánh với FR-05 và FR-12:

Tiêu chí	FR-05	FR-12	FR-08
Nền tảng test	Web	API (Postman)	API (Postman)
Số biến đầu vào	1 (String)	2 (Categorical)	4 (Mixed)
Kiểu biến	String	Categorical	String + Numeric + Implicit
BVA	Skipped	Skipped	Applied (total_amount)
Tổng test case	12	40	18 (15 DT + 3 BVA)
Pass rate	33.3%	57.5%	16.7%
Bugs phát hiện	7 (2 Critical)	4 (3 Critical)	5 (1 Critical, 4 Major)

Tiêu chí	FR-05	FR-12	FR-08
Human review loại TC	2	0	0 (nhưng BVA redundant)
Số hạn chế AI	3	2	1

FR-08 cho thấy AI Agent đặc biệt hiệu quả trong việc phân tích mâu thuẫn giữa đặc tả và API specification, xác định đúng attack vector chính (`total_amount` manipulation). Tỷ lệ fail 83.3% (cao nhất trong 3 FR) cho thấy module Checkout có vấn đề nghiêm trọng cần được fix trước khi release. Tuy nhiên, **human review vẫn là bắt buộc** để:

- Phát hiện và loại bỏ sự trùng lặp giữa DT và BVA test case
- Đánh giá ý nghĩa thực tế của BVA khi hành vi không phân biệt tại biên
- Xác nhận severity phù hợp với ngữ cảnh hệ thống (Critical cho financial manipulation)

Pool C

FR-12: Kiểm soát truy cập (Access Control)

1. Tổng quan

FR-12 yêu cầu phân hệ Admin của hệ thống EShop chỉ dành cho tài khoản có `role = 'admin'`. Cụ thể, **tất cả** các API Admin (`/api/admin/*`) và các API có tính ảnh hưởng dữ liệu (`POST/PUT/DELETE /api/products, /api/categories, /api/coupons`) đều phải yêu cầu:

1. Token JWT hợp lệ (SEC-02).
2. `role = 'admin'` trong Token (SEC-03).

Các tài liệu đặc tả được sử dụng làm cơ sở thiết kế test case:

- [description_project.md](#) -- Mục FR-12 (dòng 174-179) và SEC-02, SEC-03 (dòng 274-285)
- [api_specification.md](#) -- Mục 6: API dành cho Admin (dòng 171-214)

2. Domain Testing

2.1. Quy trình áp dụng kỹ thuật Domain Testing

Kỹ thuật Domain Testing được áp dụng theo quy trình 3 bước như sau:

Bước 1 -- Xác định biến đầu vào (Input Variables)

Từ phân tích đặc tả FR-12 trong `description_project.md` và `api_specification.md`, xác định được **2 biến đầu vào chính** và **1 biến ngữ cảnh**:

#	Biến	Kiểu	Mô tả
V1	Token (JWT)	Categorical / State	Trạng thái của JWT Token gửi kèm trong header <code>Authorization: Bearer <token></code>

#	Biến	Kiểu	Mô tả
V2	Role (trong Token)	Categorical	Giá trị role được encode trong JWT payload
V3	API Endpoint (ngữ cảnh)	Categorical	Endpoint đang được gọi, chia thành 2 nhóm: (a) Admin-only /api/admin/* , (b) Data-mutation /api/products , /api/categories , /api/coupons với method POST/PUT/DELETE

Bước 2 -- Phân hoạch tương đương (Equivalence Partitioning)

Áp dụng EP cho từng biến đầu vào:

Biến V1: Token (JWT)

Partition ID	Phân hoạch	Giá trị đại diện	Valid / Invalid
V1-EP1	Không có Token	Header Authorization trống hoặc không gửi	Invalid
V1-EP2	Token sai định dạng / hết hạn / bị giả mạo	"Bearer invalid_token_xyz"	Invalid
V1-EP3	Token JWT hợp lệ (của user thường, role = 'user')	Token từ login test@eshop.com	Valid (nhưng role sai, truy cập bị từ chối)
V1-EP4	Token JWT hợp lệ (của admin, role = 'admin')	Token từ login admin@eshop.com	Valid

Biến V2: Role (trong Token)

Partition ID	Phân hoạch	Giá trị đại diện	Valid / Invalid
V2-EP1	Không có role (Token missing hoặc invalid)	N/A -- phụ thuộc V1-EP1, V1-EP2	Invalid
V2-EP2	role = 'user' (user thường)	Token của test@eshop.com	Invalid (cho API Admin)
V2-EP3	role = 'admin'	Token của admin@eshop.com	Valid

Biến V3: API Endpoint (2 nhóm target)

Nhóm A -- Admin-only APIs (**/api/admin/***):

API	Method	Mô tả
/api/admin/users	GET	Lấy danh sách người dùng
/api/admin/users/:id	DELETE	Xóa người dùng
/api/admin/orders	GET	Lấy danh sách đơn hàng

API	Method	Mô tả
/api/admin/orders/:id/status	PUT	Cập nhật trạng thái đơn hàng
/api/admin/import-products	POST	Import sản phẩm từ CSV
/api/admin/coupons	POST	Thêm mã giảm giá
/api/admin/coupons/:id	DELETE	Xóa mã giảm giá

Nhóm B -- Data-mutation APIs (Non-admin path, nhưng yêu cầu admin):

API	Method	Mô tả
/api/products	POST	Thêm sản phẩm
/api/products/:id	PUT	Sửa sản phẩm
/api/products/:id	DELETE	Xóa sản phẩm
/api/categories	POST	Thêm danh mục
/api/categories/:id	PUT	Sửa danh mục
/api/categories/:id	DELETE	Xóa danh mục

Bước 3 -- Tổng hợp Domain Matrix và tạo Test Case

Logic tổ hợp cốt lõi từ Token x Role x API Group tạo ra 8 kịch bản logic:

#	Token (V1)	Role (V2)	API Group (V3)	Expected Result
1	Không có Token	N/A	Nhóm A (Admin API)	Bị từ chối (401 Unauthorized)
2	Không có Token	N/A	Nhóm B (Data-mutation API)	Bị từ chối (401 Unauthorized)
3	Token sai / hết hạn	N/A	Nhóm A (Admin API)	Bị từ chối (401 Unauthorized)
4	Token sai / hết hạn	N/A	Nhóm B (Data-mutation API)	Bị từ chối (401 Unauthorized)
5	Token hợp lệ	role = 'user'	Nhóm A (Admin API)	Bị từ chối (403 Forbidden)
6	Token hợp lệ	role = 'user'	Nhóm B (Data-mutation API)	Bị từ chối (403 Forbidden)
7	Token hợp lệ	role = 'admin'	Nhóm A (Admin API)	Truy cập thành công (200 OK)
8	Token hợp lệ	role = 'admin'	Nhóm B (Data-mutation API)	Truy cập thành công (200 OK)

Để đảm bảo coverage cho tất cả 13 endpoint trong cả 2 nhóm, mỗi endpoint được kiểm tra với **3 kịch bản cốt lõi**: (1) Không có Token -- 401, (2) Token hợp lệ nhưng role = 'user' -- 403, (3) Token hợp lệ + role = 'admin' -- 200 OK. Kịch bản "Token sai/hết hạn" được test đại diện trên 1 endpoint vì hành vi là đồng nhất trên tất cả endpoint.

Tổng cộng: **40 test case** (DT-001 đến DT-040).

Ma trận test case đầy đủ:

TC ID	Endpoint	Method	Token	Role	Expected
DT-001	/api/admin/users	GET	Không gửi	N/A	401
DT-002	/api/admin/users	GET	Token sai	N/A	401
DT-003	/api/admin/users	GET	Hợp lệ	user	403
DT-004	/api/admin/users	GET	Hợp lệ	admin	200
DT-005	/api/admin/users/:id	DELETE	Không gửi	N/A	401
DT-006	/api/admin/users/:id	DELETE	Hợp lệ	user	403
DT-007	/api/admin/users/:id	DELETE	Hợp lệ	admin	200
DT-008	/api/admin/orders	GET	Không gửi	N/A	401
DT-009	/api/admin/orders	GET	Hợp lệ	user	403
DT-010	/api/admin/orders	GET	Hợp lệ	admin	200
DT-011	/api/admin/orders/:id/status	PUT	Không gửi	N/A	401
DT-012	/api/admin/orders/:id/status	PUT	Hợp lệ	user	403
DT-013	/api/admin/orders/:id/status	PUT	Hợp lệ	admin	200
DT-014	/api/admin/import-products	POST	Không gửi	N/A	401
DT-015	/api/admin/import-products	POST	Hợp lệ	user	403
DT-016	/api/admin/import-products	POST	Hợp lệ	admin	200
DT-017	/api/admin/coupons	POST	Không gửi	N/A	401
DT-018	/api/admin/coupons	POST	Hợp lệ	user	403
DT-019	/api/admin/coupons	POST	Hợp lệ	admin	200
DT-020	/api/admin/coupons/:id	DELETE	Không gửi	N/A	401
DT-021	/api/admin/coupons/:id	DELETE	Hợp lệ	user	403
DT-022	/api/admin/coupons/:id	DELETE	Hợp lệ	admin	200
DT-023	/api/products	POST	Không gửi	N/A	401
DT-024	/api/products	POST	Hợp lệ	user	403

TC ID	Endpoint	Method	Token	Role	Expected
DT-025	/api/products	POST	Hợp lệ	admin	200
DT-026	/api/products/:id	PUT	Không gửi	N/A	401
DT-027	/api/products/:id	PUT	Hợp lệ	user	403
DT-028	/api/products/:id	PUT	Hợp lệ	admin	200
DT-029	/api/products/:id	DELETE	Không gửi	N/A	401
DT-030	/api/products/:id	DELETE	Hợp lệ	user	403
DT-031	/api/products/:id	DELETE	Hợp lệ	admin	200
DT-032	/api/categories	POST	Không gửi	N/A	401
DT-033	/api/categories	POST	Hợp lệ	user	403
DT-034	/api/categories	POST	Hợp lệ	admin	200
DT-035	/api/categories/:id	PUT	Không gửi	N/A	401
DT-036	/api/categories/:id	PUT	Hợp lệ	user	403
DT-037	/api/categories/:id	PUT	Hợp lệ	admin	200
DT-038	/api/categories/:id	DELETE	Không gửi	N/A	401
DT-039	/api/categories/:id	DELETE	Hợp lệ	user	403
DT-040	/api/categories/:id	DELETE	Hợp lệ	admin	200

2.2. Kết quả thực thi

TC ID	Tên Test Case	Kết quả	Ghi chú
DT-001	Truy cập API danh sách người dùng -- Không có Token	Passed	401 Unauthorized
DT-002	Truy cập API danh sách người dùng -- Token không hợp lệ	Failed	403 thay vì 401
DT-003	Truy cập API danh sách người dùng -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-004	Truy cập API danh sách người dùng -- Token admin	Passed	200 OK
DT-005	Xóa người dùng (Admin) -- Không có Token	Passed	401 Unauthorized
DT-006	Xóa người dùng (Admin) -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role

TC ID	Tên Test Case	Kết quả	Ghi chú
DT-007	Xóa người dùng (Admin) -- Token admin	Passed	200 OK
DT-008	Truy cập API danh sách đơn hàng (Admin) -- Không có Token	Passed	401 Unauthorized
DT-009	Truy cập API danh sách đơn hàng (Admin) -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-010	Truy cập API danh sách đơn hàng (Admin) -- Token admin	Passed	200 OK
DT-011	Cập nhật trạng thái đơn hàng (Admin) -- Không có Token	Passed	401 Unauthorized
DT-012	Cập nhật trạng thái đơn hàng (Admin) -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-013	Cập nhật trạng thái đơn hàng (Admin) -- Token admin	Passed	200 OK
DT-014	Import sản phẩm CSV (Admin) -- Không có Token	Passed	401 Unauthorized
DT-015	Import sản phẩm CSV (Admin) -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-016	Import sản phẩm CSV (Admin) -- Token admin	Passed	200 OK
DT-017	Thêm mã giảm giá (Admin) -- Không có Token	Passed	401 Unauthorized
DT-018	Thêm mã giảm giá (Admin) -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-019	Thêm mã giảm giá (Admin) -- Token admin	Passed	200 OK
DT-020	Xóa mã giảm giá (Admin) -- Không có Token	Passed	401 Unauthorized
DT-021	Xóa mã giảm giá (Admin) -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-022	Xóa mã giảm giá (Admin) -- Token admin	Passed	200 OK
DT-023	Thêm sản phẩm -- Không có Token	Failed	200 thay vì 401 -- hoàn toàn thiếu auth

TC ID	Tên Test Case	Kết quả	Ghi chú
DT-024	Thêm sản phẩm -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-025	Thêm sản phẩm -- Token admin	Passed	200 OK
DT-026	Cập nhật sản phẩm -- Không có Token	Failed	200 thay vì 401 -- hoàn toàn thiếu auth
DT-027	Cập nhật sản phẩm -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-028	Cập nhật sản phẩm -- Token admin	Passed	200 OK
DT-029	Xóa sản phẩm -- Không có Token	Failed	200 thay vì 401 -- hoàn toàn thiếu auth
DT-030	Xóa sản phẩm -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-031	Xóa sản phẩm -- Token admin	Passed	200 OK
DT-032	Thêm danh mục -- Không có Token	Passed	401 Unauthorized
DT-033	Thêm danh mục -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-034	Thêm danh mục -- Token admin	Passed	200 OK
DT-035	Cập nhật danh mục -- Không có Token	Passed	401 Unauthorized
DT-036	Cập nhật danh mục -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-037	Cập nhật danh mục -- Token admin	Passed	200 OK
DT-038	Xóa danh mục -- Không có Token	Passed	401 Unauthorized
DT-039	Xóa danh mục -- Token user thường	Failed	200 thay vì 403 -- thiếu kiểm tra role
DT-040	Xóa danh mục -- Token admin	Passed	200 OK

Thống kê: 23 Passed (57.5%) / 17 Failed (42.5%) trên tổng số 40 test case.

2.3. Các lỗi phát hiện từ Domain Testing

Bug ID	TC liên quan	Mô tả ngắn	Mức độ
BUG-FR12-001	DT-023, DT-024, DT-026, DT-027, DT-029, DT-030	Product API endpoints (POST/PUT/DELETE /api/products) hoàn toàn thiếu middleware xác thực -- bất kỳ ai cũng có thể thêm/sửa/xóa sản phẩm mà không cần đăng nhập	Critical
BUG-FR12-002	DT-003, DT-006, DT-009, DT-012, DT-015, DT-018, DT-021	Admin API endpoints (/api/admin/*) thiếu kiểm tra role -- user thường có Token hợp lệ có thể truy cập tất cả Admin API, vi phạm SEC-03	Critical
BUG-FR12-003	DT-033, DT-036, DT-039	Category API endpoints (POST/PUT/DELETE /api/categories) thiếu kiểm tra role -- user thường có thể tạo/sửa/xóa danh mục	Critical
BUG-FR12-004	DT-002	Token không hợp lệ trả về HTTP 403 Forbidden thay vì 401 Unauthorized -- sai mã lỗi HTTP theo chuẩn	Minor

2.4. Phân loại lỗi theo nhóm endpoint

Phân tích kết quả test cho thấy 3 mẫu lỗi (pattern) rõ ràng:

Nhóm A -- Admin-only APIs (**/api/admin/***):

- Kiểm tra Token: DAT (tất cả endpoint đều trả 401 khi không có Token)
- Kiểm tra Role: KHONG DAT (tất cả 7 endpoint đều cho phép user thường truy cập)
- Middleware chỉ xác thực sự tồn tại của Token mà không kiểm tra role = 'admin'

Nhóm B -- Product APIs (**/api/products** POST/PUT/DELETE):

- Kiểm tra Token: KHONG DAT (hoàn toàn thiếu middleware xác thực)
- Kiểm tra Role: KHONG DAT
- Lỗi hổng nghiêm trọng nhất: bất kỳ ai không cần đăng nhập đều có thể thao tác dữ liệu sản phẩm

Nhóm B -- Category APIs (**/api/categories** POST/PUT/DELETE):

- Kiểm tra Token: DAT (trả 401 khi không có Token)
- Kiểm tra Role: KHONG DAT (user thường có thể tạo/sửa/xóa danh mục)

3. Boundary Value Analysis (BVA)

3.1. Quy trình áp dụng kỹ thuật BVA

Kỹ thuật Boundary Value Analysis (BVA) được áp dụng theo quy tắc nghiêm ngặt (STRICT BVA RULE) được định nghĩa trong file [CLAUDE.md](#):

BVA chỉ được áp dụng cho các biến số (numerical variables) như price, quantity, total_amount. KHÔNG được ép hoặc suy diễn BVA trên các biến phi số như String (search queries, emails), Categorical data (roles, statuses), hoặc UI/DOM properties.

Bước 1 -- Đánh giá khả năng áp dụng BVA

Xét tất cả biến đầu vào của FR-12:

Biến	Kiểu	Có phải biến số (numerical) không?	Áp dụng BVA?
Token (JWT)	Categorical / State	Không	Không
Role (trong Token)	Categorical	Không	Không
API Endpoint	Categorical	Không	Không

Bước 2 -- Kết luận

FR-12 có 2 biến đầu vào chính (Token và Role) và 1 biến ngữ cảnh (API Endpoint), tất cả đều thuộc kiểu **categorical** (phân loại), không phải numerical (số). Theo STRICT BVA RULE, BVA chỉ áp dụng cho biến số. Do đó:

"No numerical variables found. BVA is skipped."

Tất cả biến đầu vào đã được bao phủ đầy đủ bởi kỹ thuật Equivalence Partitioning (EP) với 4 phân hoạch cho Token và 3 phân hoạch cho Role, kết hợp với 13 endpoint tạo thành 40 test case.

3.2. Giải thích lý do không áp dụng BVA cho FR-12

Trong bối cảnh FR-12, chức năng kiểm soát truy cập hoạt động dựa trên 2 cơ chế:

- **Authentication (xác thực):** Kiểm tra sự tồn tại và tính hợp lệ của JWT Token -- đây là biến trạng thái (có/không có, hợp lệ/không hợp lệ), không phải biến số.
- **Authorization (phân quyền):** Kiểm tra giá trị role trong Token -- đây là biến phân loại (admin/user), không phải biến số.

Không tồn tại ranh giới số học nào có thể áp dụng BVA:

- Token không có "giá trị biên" theo nghĩa số học -- nó hoặc hợp lệ hoặc không.
- Role là categorical với 2 giá trị rời rạc (admin, user) -- không có phổ liên tục để xác định điểm biên.
- API Endpoint là danh mục cố định -- không có thứ tự hay ranh giới số.

Do đó, việc áp dụng BVA cho FR-12 là không phù hợp. Kỹ thuật EP đã cung cấp đủ coverage cần thiết.

4. Quy trình áp dụng CLAUDE.md cho AI Agent để tạo test case

4.1. Giới thiệu về CLAUDE.md

File [CLAUDE.md](#) là file cấu hình hướng dẫn cho AI Agent (Antigravity - Claude Opus 4.6 Thinking) hoạt động như một ISTQB-Certified QA Test Designer. Đối với FR-12, các thành phần chính của CLAUDE.md được áp dụng bao gồm:

- **Vai trò:** QA Test Designer chuyên về Black-Box Testing
- **Ràng buộc:** Hành động từng bước, dừng lại và chờ phê duyệt sau mỗi bước
- **Nguồn dữ liệu:** Chỉ dựa trên [description_project.md](#) và [api_specification.md](#)
- **Quy tắc BVA:** STRICT BVA RULE -- chỉ áp dụng cho biến số (numerical)
- **Template sử dụng:** Template 1 (Domain Testing) cho tất cả 40 test case
- **Workflow:** 5 bước từ phân tích đến báo cáo lỗi

4.2. Quy trình thực hiện chi tiết

Quy trình áp dụng CLAUDE.md được thực hiện qua **3 giai đoạn chính**:

Giai đoạn 1: Phân tích và Thiết kế (Steps 1-2-3 trong CLAUDE.md)

1. Người dùng cung cấp prompt khởi động quá trình QA với cấu hình cụ thể:

- [\[FR-DIR\]](#) = [FR-12-access](#)
- [\[FR-ID\]](#) = [FR12](#)
- Chỉ định biến đầu vào có thể là token hoặc role hoặc cả 2
- Yêu cầu áp dụng STRICT BVA RULE

2. AI Agent đọc và phân tích 2 file đặc tả:

- [description_project.md](#) tại mục FR-12 (dòng 174-179) và SEC-02, SEC-03 (dòng 274-285)
- [api_specification.md](#) tại mục 6: API dành cho Admin (dòng 171-214)

3. AI Agent xác định:

- 2 biến đầu vào chính: Token (JWT) -- Categorical, Role -- Categorical
- 1 biến ngữ cảnh: API Endpoint -- 13 endpoint chia 2 nhóm
- 4 phân hoạch cho Token (V1-EP1 đến V1-EP4)
- 3 phân hoạch cho Role (V2-EP1 đến V2-EP3)

4. AI Agent đánh giá khả năng áp dụng BVA theo STRICT BVA RULE. Kết luận: cả Token và Role đều là categorical, không phải numerical -- BVA bị bỏ qua.

5. AI Agent tổng hợp Domain Matrix: 13 endpoint x 3 kịch bản cốt lõi + 1 kịch bản Token sai = **40 test case dự kiến**.

6. AI Agent trình bày bảng phân tích logic và dừng lại chờ người dùng phê duyệt.

Kết quả giai đoạn này được lưu tại: [implementation_plan_FR12.md](#)

Giai đoạn 2: Tạo Test Case (Step 4 trong CLAUDE.md)

1. Sau khi người dùng phê duyệt bảng phân tích, AI Agent tạo 40 file test case theo **Template 1 (Domain Testing)** được định nghĩa trong CLAUDE.md.

2. AI Agent sử dụng 4 subagent song song để tăng tốc quá trình sinh file:

- Group 1 (DT-001 đến DT-010): [/api/admin/users](#) GET, DELETE + [/api/admin/orders](#) GET
- Group 2 (DT-011 đến DT-022): [/api/admin/orders/:id/status](#) PUT, [/api/admin/import-products](#) POST, [/api/admin/coupons](#) POST/DELETE

- Group 3 (DT-023 đến DT-031): `/api/products` POST/PUT/DELETE
 - Group 4 (DT-032 đến DT-040): `/api/categories` POST/PUT/DELETE
3. AI Agent xác minh kết quả: tất cả 40/40 files được tạo thành công, đúng chuẩn Template 1 với đầy đủ: file path comment, Domain Analysis, Domain Matrix, Preconditions, Test data, Test steps, Expected result, Status = **Not Run**.
4. AI Agent dừng lại và yêu cầu người dùng thực thi test case trên hệ thống thực.

Giai đoạn 3: Thực thi, Human Review, và Báo cáo (Step 5 trong CLAUDE.md)

1. Người dùng thực thi 40 test case trên hệ thống EShop thực tế bằng công cụ API testing (Postman/cURL) và cập nhật actual result + status cho từng file.
2. Trong quá trình manual testing, người dùng nhận xét về AI Gap Analysis:
- AI sắp xếp test case xen kẽ giữa admin và user, khiến tester phải cập nhật access-token liên tục thay vì chạy hết tất cả test cùng một role rồi mới chuyển sang role khác.
3. Người dùng báo cáo kết quả: 23 Passed / 17 Failed.
4. AI Agent nhận kết quả và triển khai Step 5:
- Tạo `FR-12-access-run.md`: tổng hợp kết quả test run
 - Tạo 4 bug reports: `BUG-FR12-001` đến `BUG-FR12-004`
 - Tạo `FR-12-access-gap-analysis.md`: phân tích hạn chế của AI

4.3. Sơ đồ quy trình tổng quát



```

4) | |--- Dispatch 4 subagent song song (Step
| |
| |--- Tạo 40 test case files
| |--- Xác minh 40/40 files đúng template
| |
|-- Yêu cầu manual testing -----|
|
|--- Thực thi test trên SUT ----->|
    (Postman/cURL, 40 API calls)
|
|--- Báo cáo kết quả:
|     23 Passed / 17 Failed
|     + Nhận xét: test ordering
|         không tối ưu -----> |
|
| |--- Tạo Test Run summary (Step 5)
| |--- Tạo 4 Bug Reports (3 Critical + 1
Minor)|
| |--- Tạo AI Gap Analysis
| |
|-- Hoàn thành, yêu cầu commit ----|
|
|--- git add + git commit ----->|
    (Commit: 1241f5e)
```

4.5. Phân tích AI Gap Analysis

Trong quá trình sử dụng AI Agent theo CLAUDE.md cho FR-12, 2 hạn chế chính được xác định:

Hạn chế 1: Sắp xếp test case không tối ưu cho quy trình thực thi (Test Execution Order)

- AI sắp xếp test case theo endpoint (logical grouping), xen kẽ giữa các kịch bản No Token / User Token / Admin Token cho mỗi endpoint.
- Hệ quả: Tester phải liên tục chuyển đổi access token giữa các lần test (đăng nhập/đăng xuất, copy token).
- Ví dụ: DT-003 (user token) rồi DT-004 (admin token) rồi DT-005 (no token) rồi DT-006 (user token).
- Giải pháp tốt hơn: nhóm theo role/token (execution grouping) -- chạy hết tất cả "No Token" tests trước, rồi tất cả "User Token" tests, rồi "Admin Token" tests.

Hạn chế 2: Thiếu khả năng đánh giá dependency giữa các test case

- AI không thể dự đoán rằng DT-007 (xóa user bằng admin token) có thể ảnh hưởng đến DT-030 (xóa sản phẩm bằng user token) nếu user đã bị xóa.
- Nguyên nhân: Black-box testing -- AI không có thông tin về trạng thái CSDL sau mỗi lần thực thi test case.
- Giải pháp: AI nên thêm ghi chú về dependency giữa test case, đặc biệt với các thao tác DELETE có tính phá hủy dữ liệu.

4.6. Đánh giá tổng thể

Tiêu chí	Đánh giá
Số lượng test case AI tạo	40
Số lượng test case sau human review	40 (không loại bỏ)
Số lượng test case phát hiện lỗi	17/40 (42.5%)
Số lượng bug phát hiện	4 (3 Critical, 1 Minor)
Độ chính xác của test design	Rất cao -- phủ được tất cả 13 endpoint với 3 kịch bản access control
Cần chỉnh sửa bởi người dùng	Thứ tự thực thi test case (nhóm theo role thay vì endpoint)

So sánh với FR-05:

Tiêu chí	FR-05	FR-12
Số biến đầu vào	1 (String)	2 (Categorical) + 1 ngữ cảnh
Kiểu biến	String	Categorical / State
BVA	Skipped (no numerical)	Skipped (no numerical)
Tổng test case	12 (sau review)	40
Pass rate	33.3%	57.5%
Bugs phát hiện	7 (2 Critical)	4 (3 Critical)
Human review loại TC	2 test case	0 test case

AI Agent chứng minh hiệu quả đặc biệt cao trong việc thiết kế test case cho Access Control -- một lĩnh vực yêu cầu tính hệ thống và coverage toàn diện trên nhiều endpoint. Tuy nhiên, **human review vẫn là bắt buộc** để:

- Tối ưu hóa thứ tự thực thi test case theo workflow thực tế của tester
- Xác định dependency giữa các test case có thao tác phá hủy dữ liệu
- Đánh giá mức độ nghiêm trọng (severity) phù hợp với ngữ cảnh hệ thống thực tế

Pool D

FR-09: Mã Giảm Giá (Coupon) -- Mobile App

1. Tổng quan

FR-09 định nghĩa chức năng mã giảm giá (Coupon) tại bước Checkout của hệ thống EShop. Người dùng có thể nhập mã giảm giá và hệ thống áp dụng giảm giá dựa trên **5 điều kiện** sau, tất cả phải thỏa mãn:

#	Điều kiện	Mô tả
C1	Mã tồn tại	Mã phải có trong CSDL và đang hoạt động (is_active = 1)

#	Điều kiện	Mô tả
C2	Còn hạn sử dụng	Ngày hiện tại phải trước <code>expired_at</code>
C3	Đủ ngưỡng đơn hàng	Tổng đơn hàng $\geq$ (lớn hơn hoặc bằng) <code>min_order_amount</code>
C4	Đã đăng nhập	Người dùng phải có JWT Token hợp lệ
C5	Chưa dùng hết lượt	Số lần đã dùng mã này của user $<$ <code>max_uses_per_user</code>

Công thức tính giảm giá:

- Loại `percent`: `discount_amount = total x discount_value / 100`
- Loại `fixed`: `discount_amount = discount_value`
- `final_amount = total - discount_amount`

Mã giảm giá mẫu trong hệ thống:

Mã	Loại	Giá trị	Ngưỡng tối thiểu	Hạn sử dụng	Số lần/người
SAVE10	percent	10%	300,000 đồng	2099-12-31	1
BIGBUY	fixed	50,000 đồng	500,000 đồng	2099-12-31	1
VIP100	fixed	100,000 đồng	300,000 đồng	2099-12-31	2
EXPIRED	percent	20%	100,000 đồng	2020-01-01	1

Các tài liệu đặc tả được sử dụng làm cơ sở thiết kế test case:

- [description_project.md](#) -- Mục FR-09 (dòng 110-136)
- [api_specification.md](#) -- Endpoint `POST /api/apply-coupon`

Môi trường kiểm thử: Mobile App (React Native + Expo) trên thiết bị di động, Backend tại <http://localhost:3000>.

2. Domain Testing

2.1. Quy trình áp dụng kỹ thuật Domain Testing

Kỹ thuật Domain Testing được áp dụng theo quy trình 3 bước như sau:

Bước 1 -- Xác định biến đầu vào (Input Variables)

Từ phân tích FR-09 và 5 điều kiện (C1-C5), xác định được **5 biến đầu vào**:

#	Biến	Kiểu	Nguồn	Miền giá trị / Ràng buộc
V1	<code>code</code> (Mã giảm giá)	String (Input field)	Người dùng nhập trên Mobile App	Mã phải tồn tại trong CSDL, đang hoạt động (C1)
V2	<code>total_amount</code> (Tổng đơn hàng)	Numeric (Implicit)	Tính từ giỏ hàng	Tổng đơn hàng $\geq$ <code>min_order_amount</code> của coupon

#	Biến	Kiểu	Nguồn	Miền giá trị / Ràng buộc
				(C3)
V3	Authorization (JWT Token)	String (Header)	HTTP Header	Token JWT hợp lệ từ người dùng đã đăng nhập (C4)
V4	expired_at (Hạn sử dụng)	Date (Server-side)	CSDL	Ngày hiện tại phải trước expired_at (C2)
V5	usage_count (Số lần đã dùng)	Numeric (Server-side)	CSDL	Số lần đã dùng < max_uses_per_user (C5)

Bước 2 -- Phân hoạch tương đương (Equivalence Partitioning)

Áp dụng EP cho từng biến, kết hợp với 4 mã giảm giá mẫu (SAVE10, BIGBUY, VIP100, EXPIRED):

EP ID	Biến	Partition	Giá trị mẫu	Expected
EP-V1-01	code	Valid -- SAVE10 (percent)	"SAVE10"	Thành công, giảm 10%
EP-V1-02	code	Valid -- BIGBUY (fixed)	"BIGBUY"	Thành công, giảm 50,000 đồng
EP-V1-03	code	Valid -- VIP100 (fixed, max=2)	"VIP100"	Thành công, giảm 100,000 đồng
EP-V1-04	code	Invalid -- Mã không tồn tại	"FAKECODE"	Từ chối
EP-V1-05	code	Invalid -- Chuỗi rỗng	""	Từ chối
EP-V1-06	code	Invalid -- Sai case (lowercase)	"save10"	Test case-sensitivity
EP-V2-01	total_amount	Valid -- Trên ngưỡng	500,000 đồng	Chấp nhận
EP-V2-02	total_amount	Invalid -- Dưới ngưỡng	200,000 đồng (< 300,000 đồng)	Từ chối
EP-V3-01	Authorization	Valid -- Đã đăng nhập	Token hợp lệ	Cho phép
EP-V3-02	Authorization	Invalid -- Chưa đăng nhập	Không có token	Từ chối
EP-V4-01	expired_at	Valid -- Còn hạn	2099-12-31	Chấp nhận
EP-V4-02	expired_at	Invalid -- Hết hạn	EXPIRED (2020-01-01)	Từ chối

EP ID	Biến	Partition	Giá trị mẫu	Expected
EP-V5-01	usage_count	Valid -- Chưa dùng hết	0 < 1	Cho phép
EP-V5-02	usage_count	Invalid -- Đã dùng hết	1 >= 1 (SAVE10)	Từ chối

Bước 3 -- Tổng hợp Domain Matrix và tạo Test Case

Tổng hợp thành **10 test case Domain Testing** (DT-001 đến DT-010):

TC ID	Điều kiện test	Mã	Mô tả
DT-001	Tất cả C1-C5 hợp lệ	SAVE10	Áp dụng thành công -- loại percent (10%)
DT-002	Tất cả C1-C5 hợp lệ	BIGBUY	Áp dụng thành công -- loại fixed (50,000 đồng)
DT-003	Tất cả C1-C5 hợp lệ	VIP100	Áp dụng thành công -- loại fixed (100,000 đồng, max=2)
DT-004	C1 vi phạm	FAKECODE	Mã không tồn tại trong CSDL
DT-005	C1 vi phạm	(rỗng)	Chuỗi rỗng -- không nhập gì
DT-006	C2 vi phạm	EXPIRED	Mã hết hạn sử dụng (2020-01-01)
DT-007	C3 vi phạm	SAVE10	Tổng đơn dưới ngưỡng (200,000 đồng < 300,000 đồng)
DT-008	C4 vi phạm	--	Người dùng chưa đăng nhập
DT-009	C5 vi phạm	SAVE10	Đã dùng hết lượt (usage=1, max=1)
DT-010	Case-sensitivity	save10	Nhập mã lowercase -- kiểm tra hệ thống có phân biệt hoa thường

2.2. Kết quả thực thi Domain Testing

TC ID	Tên Test Case	Kết quả	Bug ID
DT-001	Áp dụng SAVE10 thành công (percent)	Failed	BUG-FR09-001
DT-002	Áp dụng BIGBUY thành công (fixed)	Passed	--
DT-003	Áp dụng VIP100 thành công (fixed, max=2)	Passed	--

TC ID	Tên Test Case	Kết quả	Bug ID
DT-004	Mã không tồn tại (FAKECODE)	Passed	--
DT-005	Chuỗi mã rỗng	Passed	--
DT-006	Mã hết hạn sử dụng (EXPIRED)	Passed	--
DT-007	Tổng đơn dưới ngưỡng tối thiểu	Passed	--
DT-008	Người dùng chưa đăng nhập	Passed	--
DT-009	Đã dùng hết lượt cho phép	Passed	--
DT-010	Mã đúng nhưng nhập sai case (lowercase "save10")	Failed	BUG-FR09-001

Thống kê Domain Testing: 8 Passed (80.0%) / 2 Failed (20.0%) trên tổng số 10 test case.

Ghi chú: Cả 2 test case Failed đều liên quan đến mã giảm giá loại **percent** (SAVE10). Mã loại **fixed** (BIGBUY, VIP100) hoạt động đúng. DT-010 bản chất là test case-sensitivity nhưng cũng bị lỗi tính giá do SAVE10 là loại percent.

2.3. Các lỗi phát hiện từ Domain Testing

Bug ID	TC liên quan	Mô tả ngắn	Mức độ
BUG-FR09-001	DT-001, DT-010, BVA-003, BVA-010	Lỗi tính giảm giá loại percent -- hệ thống tính sai gấp 100 lần. Ví dụ: SAVE10 (10%), tổng 500,000 đồng, hệ thống tính discount = 5,000,000 đồng thay vì 50,000 đồng. Công thức sai: total x discount_value thay vì total x discount_value / 100	Critical
BUG-FR09-002	BVA-002, BVA-005	Lỗi biên off-by-one -- min_order_amount dùng phép so sánh > thay vì >=. Khi tổng đơn = đúng ngưỡng tối thiểu, hệ thống từ chối thay vì chấp nhận. Vi phạm trực tiếp đặc tả FR-09 Điều kiện C3	Major

3. Boundary Value Analysis (BVA)

3.1. Quy trình áp dụng kỹ thuật BVA

Kỹ thuật Boundary Value Analysis (BVA) được áp dụng theo quy tắc nghiêm ngặt (STRICT BVA RULE) được định nghĩa trong file [CLAUDE.md](#):

BVA chỉ được áp dụng cho các biến số (numerical variables). KHÔNG được ép hoặc suy diễn BVA trên các biến phi số.

Bước 1 -- Đánh giá khả năng áp dụng BVA

Xét tất cả biến đầu vào của FR-09:

Biến	Kiểu	Có phải biến số (numerical) không?	Áp dụng BVA?
code	String	Không	Không
total_amount	Numeric (Integer)	Có	Có
Authorization	String (Header)	Không	Không
expired_at	Date (Server-side)	Không	Không
usage_count	Numeric (Integer)	Có	Có

FR-09 có **2 biến numerical** thỏa mãn STRICT BVA RULE: `total_amount` và `usage_count`. Đây là FR có nhiều biến BVA nhất trong 5 FR được test.

Bước 2 -- Xác định Boundary cho từng biến

Biến 1: `total_amount` -- Ngưỡng tối thiểu (`min_order_amount`)

Theo đặc tả C3: Tổng đơn hàng **>= (lớn hơn hoặc bằng)** `min_order_amount`. Boundary tại giá trị `min_order_amount` của từng mã giảm giá:

Mã giảm giá	<code>min_order_amount</code>	BVA Points
SAVE10	300,000 đồng	299,999 đồng (OFF ngoài bên trái), 300,000 đồng (ON) , 300,001 đồng (OFF ngoài bên phải)
BIGBUY	500,000 đồng	499,999 đồng (OFF ngoài bên trái), 500,000 đồng (ON) , 500,001 đồng (OFF ngoài bên phải)

Biến 2: `usage_count` -- Giới hạn lượt sử dụng (`max_uses_per_user`)

Theo đặc tả C5: Số lần đã dùng **<** `max_uses_per_user`. Boundary tại giá trị `max_uses_per_user`:

Mã giảm giá	<code>max_uses_per_user</code>	BVA Points (<code>usage_count</code>)
VIP100	2	0 (OFF ngoài bên trái), 1 (ON-1) , 2 (ON -- đã hết lượt)
SAVE10	1	0 (ON -- lần đầu) , 1 (OFF ngoài bên phải -- đã hết)

Bước 3 -- Tạo BVA Test Case

Tổng hợp thành **11 test case BVA** (BVA-001 đến BVA-011):

Nhóm 1: BVA cho `total_amount` -- Mã SAVE10 (ngưỡng 300,000 đồng):

TC ID	<code>total_amount</code>	Boundary Point	Expected
BVA-001	299,999 đồng	OFF ngoài bên trái	Từ chối -- dưới ngưỡng
BVA-002	300,000 đồng	ON	Chấp nhận -- đúng ngưỡng
BVA-003	300,001 đồng	OFF ngoài bên phải	Chấp nhận -- trên ngưỡng

Nhóm 2: BVA cho **total_amount** -- Mã BIGBUY (ngưỡng 500,000 đồng):

TC ID	total_amount	Boundary Point	Expected
BVA-004	499,999 đồng	OFF ngoài bên trái	Từ chối -- dưới ngưỡng
BVA-005	500,000 đồng	ON	Chấp nhận -- đúng ngưỡng
BVA-006	500,001 đồng	OFF ngoài bên phải	Chấp nhận -- trên ngưỡng

Nhóm 3: BVA cho **usage_count** -- Mã VIP100 (max=2):

TC ID	usage_count	Boundary Point	Expected
BVA-007	0 (lần 1)	OFF ngoài bên trái	Chấp nhận
BVA-008	1 (lần 2)	ON-1	Chấp nhận (lần cuối)
BVA-009	2 (lần 3)	ON -- đã hết lượt	Từ chối

Nhóm 4: BVA cho **usage_count** -- Mã SAVE10 (max=1):

TC ID	usage_count	Boundary Point	Expected
BVA-010	0 (lần 1)	ON	Chấp nhận (lần đầu và duy nhất)
BVA-011	1 (lần 2)	OFF ngoài bên phải	Từ chối -- đã dùng hết

3.2. Kết quả thực thi BVA

TC ID	Tên Test Case	Biến test	BVA Point	Kết quả	Bug ID
BVA-001	SAVE10 -- total = 299,999 đồng	total_amount	OFF ngoài bên trái	Passed	--
BVA-002	SAVE10 -- total = 300,000 đồng	total_amount	ON	Failed	BUG-FR09-002
BVA-003	SAVE10 -- total = 300,001 đồng	total_amount	OFF ngoài bên phải	Failed	BUG-FR09-001
BVA-004	BIGBUY -- total = 499,999 đồng	total_amount	OFF ngoài bên trái	Passed	--
BVA-005	BIGBUY -- total = 500,000 đồng	total_amount	ON	Failed	BUG-FR09-002
BVA-006	BIGBUY -- total = 500,001 đồng	total_amount	OFF ngoài bên phải	Passed	--
BVA-007	VIP100 -- Sử dụng lần 1 (usage=0, max=2)	usage_count	OFF ngoài bên trái	Passed	--

TC ID	Tên Test Case	Biến test	BVA Point	Kết quả	Bug ID
BVA-008	VIP100 -- Sử dụng lần 2 (usage=1, max=2)	usage_count	ON-1	Passed	--
BVA-009	VIP100 -- Sử dụng lần 3 (usage=2, max=2)	usage_count	ON -- đã hết	Passed	--
BVA-010	SAVE10 -- Sử dụng lần 1 (usage=0, max=1)	usage_count	ON	Failed	BUG-FR09-001
BVA-011	SAVE10 -- Sử dụng lần 2 (usage=1, max=1)	usage_count	OFF ngoài bên phải	Passed	--

Thống kê BVA: 7 Passed (63.6%) / 4 Failed (36.4%) trên tổng số 11 test case.

3.3. Phân tích kết quả BVA chi tiết

Bug BUG-FR09-002 (off-by-one) -- Phát hiện bởi BVA:

total_amount gửi	So sánh với min_order_amount	Expected	Actual	Kết quả
299,999 đồng	< 300,000 đồng	Từ chối	Từ chối	Đúng
300,000 đồng	= 300,000 đồng	Chấp nhận (>=)	Từ chối (>)	SAI -- off-by-one
300,001 đồng	> 300,000 đồng	Chấp nhận	Chấp nhận	Đúng

Pattern tương tự xảy ra với BIGBUY (BVA-004/005/006 tại ngưỡng 500,000 đồng).

Đây là ví dụ điển hình của BVA phát hiện lỗi off-by-one: backend dùng phép so sánh > (strictly greater than) thay vì >= (greater than or equal) khi kiểm tra điều kiện C3. Lỗi này **chỉ có thể phát hiện bởi BVA** -- Domain Testing (EP) sẽ không phát hiện vì EP không test đúng tại điểm biên.

Bug BUG-FR09-001 (percent calculation) -- Phát hiện bởi cả DT và BVA:

BVA-003 (total = 300,001 đồng, SAVE10) và BVA-010 (usage=0, SAVE10) đều thất bại vì SAVE10 là loại percent và công thức tính sai. Bug này được phát hiện đầu tiên bởi DT-001 và xác nhận lại bởi BVA.

4. Quy trình áp dụng CLAUDE.md cho AI Agent để tạo test case

4.1. Giới thiệu về CLAUDE.md

File [CLAUDE.md](#) là file cấu hình hướng dẫn cho AI Agent (Antigravity - Claude Opus 4.6 Thinking) hoạt động như một ISTQB-Certified QA Test Designer. Đối với FR-09, các thành phần chính của CLAUDE.md được áp dụng bao gồm:

- **Vai trò:** QA Test Designer chuyên về Black-Box Testing
- **Ràng buộc:** Hành động từng bước, dừng lại và chờ phê duyệt sau mỗi bước
- **Nguồn dữ liệu:** Chỉ dựa trên `description_project.md` và `api_specification.md`
- **Quy tắc BVA:** STRICT BVA RULE -- 2 biến numerical (`total_amount`, `usage_count`) thỏa mãn -- BVA ĐƯỢC ÁP DỤNG
- **Template sử dụng:** Template 1 (Domain Testing) cho 10 TC + Template 2 (BVA) cho 11 TC
- **Workflow:** 5 bước từ phân tích đến báo cáo lỗi
- **Đặc thù FR-09:** Test trên Mobile App -- test steps sử dụng ngôn ngữ mobile (chạm, bàn phím ảo, Toast/Alert)

FR-09 là FR có nhiều test case BVA nhất (11 TC) vì có 2 biến numerical và nhiều mã giảm giá với các ngưỡng khác nhau.

4.2. Quy trình thực hiện chi tiết

Quy trình áp dụng CLAUDE.md được thực hiện qua **3 giai đoạn chính**:

Giai đoạn 1: Phân tích và Thiết kế (Steps 1-2-3 trong CLAUDE.md)

1. Người dùng cung cấp prompt khởi động quá trình QA với cấu hình cụ thể:
 - `[FR-DIR] = FR-09-coupon-mobile`
 - `[FR-ID] = FR09`
 - Chỉ định test trên nền tảng Mobile App
 - Yêu cầu áp dụng BVA cho các biến numerical (`total_amount`, `usage_count`)
2. AI Agent đọc và phân tích file đặc tả:
 - `description_project.md` tại mục FR-09 (dòng 110-136): 5 điều kiện, công thức tính giá, 4 mã mẫu
 - `api_specification.md`: endpoint `POST /api/apply-coupon`
3. AI Agent xác định:
 - 5 biến đầu vào (V1-V5) tương ứng với 5 điều kiện C1-C5
 - 14 phân vùng EP cho 5 biến
 - BVA áp dụng cho 2 biến numerical: `total_amount` (6 BVA points cho 2 mã) và `usage_count` (5 BVA points cho 2 mã)
4. AI Agent tính sẵn công thức giảm giá (percent và fixed) trong mỗi test case để giúp tester so sánh trực tiếp expected vs actual.
5. AI Agent trình bày bảng phân tích logic và dừng lại chờ phê duyệt.

Giai đoạn 2: Tạo Test Case (Step 4 trong CLAUDE.md)

1. Sau khi người dùng phê duyệt bảng phân tích, AI Agent tạo **21 file test case** (10 DT + 11 BVA) theo template CLAUDE.md.
2. AI Agent tạo file với test steps đúng ngữ cảnh Mobile App:
 - Sử dụng "chạm" (tap) thay vì "click"

- Mô tả bàn phím ảo (virtual keyboard) khi nhập mã
- Toast/Alert thay vì inline message cho phản hồi hệ thống
- Tab "Giỏ hàng" trên thanh điều hướng dưới cùng

3. AI Agent xác minh cấu trúc thư mục: **domain-testing/** (10 files) + **bva/** (11 files).

4. AI Agent dừng lại và yêu cầu người dùng thực thi test case trên thiết bị mobile thực.

Giai đoạn 3: Thực thi, Human Review, và Báo cáo (Step 5 trong CLAUDE.md)

1. Người dùng thực thi 21 test case trên thiết bị mobile thực và cập nhật actual result + status cho từng file.

2. Trong quá trình testing, người dùng phát hiện:

- Lỗi tính giảm giá percent nghiêm trọng (sai gấp 100 lần) -- phát hiện bởi DT-001
- Lỗi off-by-one tại điểm biên `min_order_amount` -- phát hiện bởi BVA-002 và BVA-005
- AI không hướng dẫn cách tạo sản phẩm giả để đạt giá trị biên chính xác (GAP-01)
- Một số test case BVA trùng lặp thao tác với DT (GAP-02)

3. Người dùng báo cáo kết quả: 14 Passed / 7 Failed.

4. AI Agent nhận kết quả và triển khai Step 5:

- Tạo [FR-09-coupon-mobile-run.md](#): tổng hợp kết quả test run
- Tạo 2 bug reports: [BUG-FR09-001.md](#) (Critical) và [BUG-FR09-002.md](#) (Major)
- Tạo [FR-09-coupon-mobile-gap-analysis.md](#): phân tích hạn chế của AI

4.3. Sơ đồ quy trình tổng quát

Người dùng	AI Agent (CLAUDE.md)
--- Prompt khởi động QA FR-09 -----> (cung cấp [FR-DIR], [FR-ID], chỉ định test trên Mobile, BVA cho total_amount và usage_count)	--- Đọc description_project.md (FR-09) --- Đọc api_specification.md --- Xác định 5 biến (V1-V5) = 5 điều
kiện C1-C5 (Step 3) (percent/fixed)	--- Phân hoạch EP: 14 partition (Step 2) --- BVA: total_amount (6 points, 2 mã) usage_count (5 points, 2 mã) --- Tính sẵn công thức giảm giá
<-- Trình bày bảng phân tích ----- (Chờ phê duyệt)	

```

| --- Phê duyệt ----->|
|
|
| --- Tạo 10 DT + 11 BVA = 21 files (Step
4)
|
| --- Test steps đúng ngữ cảnh Mobile
|      (chạm, bàn phím ảo, Toast/Alert)
|
|<--- Yêu cầu test trên Mobile -----
|      thiết bị di động thực
|
| --- Thực thi 21 test case ----->
|      trên Mobile App
|
| --- Báo cáo kết quả:
|      14 Passed / 7 Failed
|      + Percent tính sai gấp 100 lần
|      + Off-by-one tại min_order
|      + AI không hướng dẫn tạo
|      sản phẩm giả cho BVA ----->|
|
|
| --- Tạo Test Run summary (Step 5)
| --- Tạo 2 Bug Reports (1 Critical + 1
Major)
|
| --- Tạo AI Gap Analysis (2 hạn chế)
|
|<--- Hoàn thành, yêu cầu commit ----
|
| --- git add + git commit ----->|
|      (Commit: 3863ddc)

```

4.4. Phân tích AI Gap Analysis

Trong quá trình sử dụng AI Agent theo CLAUDE.md cho FR-09 trên Mobile, **2 hạn chế chính** được xác định:

Hạn chế 1: Không hướng dẫn tạo sản phẩm giả để đáp ứng giá trị biên BVA

- AI tạo test case BVA với các giá trị biên chính xác (ví dụ: 299,999 đồng, 300,000 đồng, 300,001 đồng) nhưng **không xem xét** rằng seed data trong hệ thống có các sản phẩm với giá trị quá lớn so với các ngưỡng biên cần test.
- Tester không thể tạo giỏ hàng có tổng giá trị **đúng bằng** 299,999 đồng chỉ từ sản phẩm có sẵn.
- AI cần bổ sung hướng dẫn chuẩn bị dữ liệu test trong phần Preconditions, ví dụ: "Tạo sản phẩm giả với giá 299,999 đồng qua Admin panel hoặc API POST /api/products."

Hạn chế 2: Tạo test case BVA trùng lặp thao tác với DT

- BVA-010 (SAVE10 usage=0, max=1) giống DT-001 về thao tác -- đều áp dụng SAVE10 trên tổng đơn >= 300,000 đồng.
- BVA-008 giống BVA-007 về thao tác -- đều nhập VIP100 trên cùng tổng đơn 400,000 đồng, chỉ khác trạng thái **usage_count**.
- AI nên nhận diện overlap và ghi chú rõ ràng test case nào có thể tham chiếu kết quả từ test case khác thay vì thực hiện lại.

4.5. Đánh giá tổng thể

Tiêu chí	Đánh giá
Số lượng test case AI tạo	21 (10 DT + 11 BVA)
Số lượng test case sau human review	21 (không loại bỏ)
Số lượng test case phát hiện lỗi	7/21 (33.3%)
Số lượng bug phát hiện	2 (1 Critical, 1 Major)
Độ chính xác của test design	Cao -- công thức tính sẵn giúp phát hiện bug percent, BVA 3-point phát hiện off-by-one
Cần chỉnh sửa bởi người dùng	Hướng dẫn chuẩn bị dữ liệu test, loại bỏ trùng lặp BVA/DT

So sánh với FR-05, FR-12 và FR-08:

Tiêu chí	FR-05	FR-12	FR-08	FR-09
Nền tảng test	Web	API	API	Mobile App
Số biến đầu vào	1	2	4	5
Kiểu biến	String	Categorical	Mixed	Mixed (2 numerical)
BVA	Skipped	Skipped	Applied (1 biến)	Applied (2 biến)
Số test case BVA	0	0	3	11
Tổng test case	12	40	18	21
Pass rate	33.3%	57.5%	16.7%	66.7%
Bugs phát hiện	7	4	5	2
Bug bởi BVA (riêng)	0	0	0	1 (off-by-one)
Hạn chế AI	3	2	1	2

FR-09 là FR điển hình nhất cho việc áp dụng BVA trong thực tế:

- **BVA phát hiện được bug mà Domain Testing không thể:** Lỗi off-by-one (BUG-FR09-002) chỉ có thể phát hiện khi test đúng tại điểm biên `min_order_amount`. Domain Testing với giá trị "dưới ngưỡng" (DT-007, total=200,000 đồng) và "trên ngưỡng" (DT-001, total=500,000 đồng) đều không phát hiện lỗi này.
- **Công thức tính sẵn là điểm mạnh của AI:** AI tính sẵn expected result (ví dụ: `500,000 x 10 / 100 = 50,000 đồng`) giúp tester so sánh trực tiếp và phát hiện ngay lỗi tính sai gấp 100 lần.
- **Human review vẫn bắt buộc** để chuẩn bị dữ liệu test (tạo sản phẩm giả đạt giá trị biên) và loại bỏ trùng lặp giữa DT và BVA.

Demo sử dụng AI Agent cho FR-09

Youtube demo: [Nội dung demo](#)

Drive demo: [Nội dung demo](#)